

Tiled Benchmark Analysis

- Testing with tile sizes of 8×8 , 16×16 , 32×32
 - Reasoning
 - $8 \times 8 \rightarrow 64$ threads $\rightarrow 2$ warps full
 - Above 32×32
 - 64×64 is 4096 threads/block
 - max is 1024 can't do
- Tested across matrix sizes 2048 & 4096
 - Established in naive kernel that workload was too small
 - Tiled kernel should run faster than naive so the noise from ramp-up and tail effects would be even worse
- 2048×2048
 - Best performing tile size is 32
 - cuBLAS is 33% faster
 - Most reuse compared to other tile sizes
 - $2 \times$ of 16 and $4 \times$ of 8
 - $\frac{1}{2}$ global mem pulls of 16 , $\frac{1}{4}$ of 32
 - Downside of bigger tile size
 - $\hookrightarrow$ Block has more warps
 - $\hookrightarrow$ Fewer blocks fit in 1 SM
 - $\hookrightarrow$ SM has less blocks to switch to in order to latency hide
 - $\hookrightarrow$ SM more idle
 - SM has warp limit on 5080
 - 32 tile size $\rightarrow 32$ warps/block $\rightarrow 1$ block/sM
- 2048×2048
 - Small enough that fits in L2
 - 64 mb L2 size
 - $2048^2 \times 4B \times 3$ matrices = 50.3 mb
 - Memory misses don't cost that much so even though SM has fewer blocks it doesn't have to switch as much since there's less latency to hide

- 4096×4096

- Best Performing Tile Size: 16

- Balances amount of reuse with # of blocks

- Crucially here all the data cannot fit in L2

- $4096^2 \times 4B \times 3 \text{ matrices} = 201 \text{ MB} > 64 \text{ MB}$

↳ this means have blocks to switch to

on the SM for latency hiding matters more

- since more fetches from slower DRAM

- cuBLAS is 93% faster

- Tile size 8 was worst for all cases